

6.6. PERFORMANCE ISSUES

Performance issues are very important in computer networks. When hundreds or thousands of computers are connected together, complex interactions, with unforeseen consequences, are common. Frequently, this complexity leads to poor performance and no one knows why. In the following sections, we will examine many issues related to network performance to see what kinds of problems exist and what can be done about them.

Unfortunately, understanding network performance is more of an art than a science. There is little underlying theory that is actually of any use in practice.

The best we can do is give rules of thumb gained from hard experience and present examples taken from the real world. We have intentionally delayed this discussion until after studying the transport layer in TCP and ATM networks in order to be able to point out places where they have done things right or done things wrong.

The transport layer is not the only place performance issues arise. We saw some of them in the network layer in the previous chapter. Nevertheless, the network layer tends to be largely concerned with routing and congestion control. The broader, system-oriented issues tend to be transport related, so this chapter is an appropriate place to examine them.

In the next five sections, we will look at five aspects of network performance:

1. Performance problems.
2. Measuring network performance.
3. System design for better performance.
4. Fast TPDU processing.
5. Protocols for future high-performance networks.

As an aside, we need a name for the units exchanged by transport entities. The TCP term, segment, is confusing at best and is never used outside the TCP world in this context. The proper ATM terms, CS-PDU, SAR-PDU, and CPCS-PDU, are specific to ATM. Packets clearly refer to the network layer and messages belong to the application layer. For lack of a standard term, we will go back to calling the units exchanged by transport entities TPDU. When we mean both TPDU and packet together, we will use packet as the collective term, as in "The CPU must be fast enough to process incoming packets in real time." By this we mean both the network layer packet and the TPDU encapsulated in it.

6.6.1. Performance Problems in Computer Networks

Some performance problems, such as congestion, are caused by temporary resource overloads. If more traffic suddenly arrives at a router than the router can handle, congestion will build up and performance will suffer. We studied congestion in detail in the previous chapter.

Performance also degrades when there is a structural resource imbalance. For example, if a gigabit communication line is attached to a low-end PC, the poor CPU will not be able to process the incoming packets fast enough, and some will be lost. These packets will eventually be retransmitted, adding delay, wasting bandwidth, and generally reducing performance.

Overloads can also be synchronously triggered. For example, if a TPDU contains a bad parameter (e.g., the port or process for which it is destined), in many

cases the receiver will thoughtfully send back an error notification. Now consider what could happen if a bad TPDU is broadcast to 10,000 machines: each one might send back an error message. The resulting **broadcast storm** could cripple the network. UDP suffered from this problem until the protocol was changed to cause hosts to refrain from responding to errors in UDP TPDU's sent to broadcast addresses.

A second example of synchronous overload is what happens after an electrical power failure. When the power comes back on, all the machines simultaneously jump to their ROMs to start rebooting. A typical reboot sequence might require first going to some (RARP) server to learn one's true identity, and then to some file server to get a copy of the operating system. If hundreds of machines all do this at once, the server will probably collapse under the load.

Even in the absence of synchronous overloads and when there are sufficient resources available, poor performance can occur due to lack of system tuning. For example, if a machine has plenty of CPU power and memory, but not enough of the memory has been allocated for buffer space, overruns will occur and TPDU's will be lost. Similarly, if the scheduling algorithm does not give a high enough priority to processing incoming TPDU's, some of them may be lost.

Another tuning issue is setting timeouts correctly. When a TPDU is sent, a timer is typically set to guard against its loss. If the timeout is set too short, unnecessary retransmissions will occur, clogging the wires. If the timeout is set too long, unnecessary delays will occur after a TPDU is lost. Other tunable parameters include how long to wait for data to piggyback onto before sending a separate acknowledgement and the number of retransmissions before giving up.

Gigabit networks bring with them new performance problems. Consider, for example, sending data from San Diego to Boston when the receiver's buffer is 64K bytes. Suppose that the link is 1 Gbps and the one-way speed-of-light-in-fiber delay is 20 msec. Initially, at $t = 0$, the pipe is empty, as illustrated in Fig. 6-46(a). Only 500 μ sec later, in Fig. 6-46(b), all the TPDU's are out on the fiber. The lead TPDU will now be somewhere in the vicinity of Brawley, still deep in Southern California. However, the transmitter must stop until it gets a window update.

After 20 msec, the lead TPDU hits Boston, as shown in Fig. 6-46(c) and is acknowledged. Finally, 40 msec after starting, the first acknowledgement gets back to the sender and the second burst can be transmitted. Since the transmission line was used for 0.5 msec out of 40, the efficiency is about 1.25 percent. This situation is typical of running older protocols over gigabit lines.

A useful quantity to keep in mind when analyzing network performance is the bandwidth-delay product. It is obtained by multiplying the bandwidth (in bits/sec) by the round-trip delay time (in sec). The product is the capacity of the pipe from the sender to the receiver and back (in bits).

For the example of Fig. 6-46 the bandwidth-delay product is 40 million bits. In other words, the sender would have to transmit a burst of 40 million bits to be

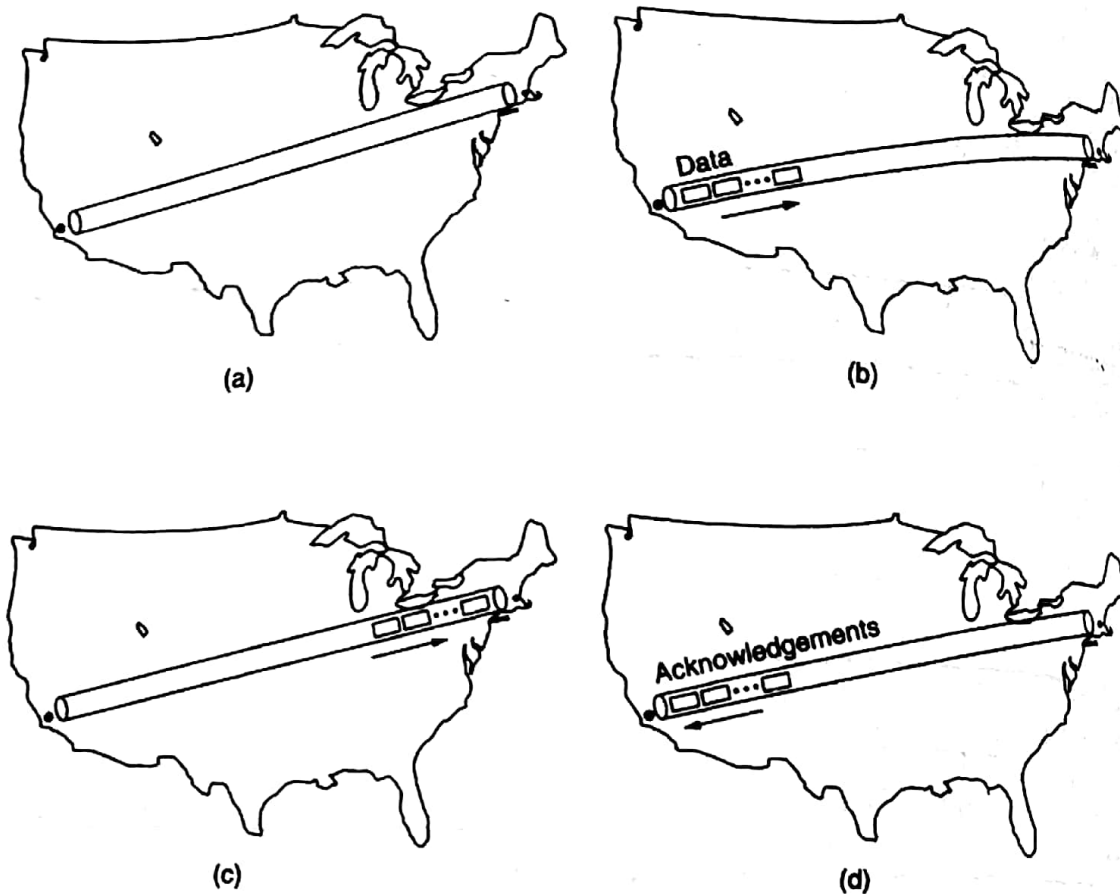


Fig. 6-46. The state of transmitting one megabit from San Diego to Boston. (a) At $t = 0$. (b) After $500 \mu\text{sec}$. (c) After 20 msec . (d) After 40 msec .

able to keep going full speed until the first acknowledgement came back. It takes this many bits to fill the pipe (in both directions). This is why a burst of half a million bits only achieves a 1.25 percent efficiency: it is only 1.25 percent of the pipe capacity.

The conclusion to be drawn here is that to achieve good performance, the receiver's window must be at least as large as the bandwidth-delay product, preferably somewhat larger since the receiver may not respond instantly. For a transcontinental gigabit line, at least 5 megabytes are required for each connection.

If the efficiency is terrible for sending a megabit, imagine what it is like when sending a few hundred bytes for a remote procedure call. Unless some other use can be found for the line while the first client is waiting for its reply, a gigabit line is no better than a megabit line, just more expensive.

Another performance problem that occurs with time-critical applications like audio and video is jitter. Having a short mean transmission time is not enough. A small standard deviation is also required. Achieving a short mean transmission time along with a small standard deviation demands a serious engineering effort.

6.6.2. Measuring Network Performance

When a network performs poorly, its users often complain to the folks running it, demanding improvements. To improve the performance, the operators must first determine exactly what is going on. To find out what is really happening, the operators must make measurements. In this section we will look at network performance measurements. The discussion below is based on the work of Mogul (1993). For a more thorough discussion of the measurement process, see (Jain, 1991; and Villamizan and Song, 1995).

The basic loop used to improve network performance contains the following steps:

- ✓ 1. Measure the relevant network parameters and performance.
2. Try to understand what is going on.
3. Change one parameter.

These steps are repeated until the performance is good enough or it is clear that the last drop of improvement has been squeezed out.

Measurements can be made in many ways and at many locations (both physically and in the protocol stack). The most basic kind of measurement is to start a timer when beginning some activity and use it to see how long that activity takes. For example, knowing how long it takes for a TPDU to be acknowledged is a key measurement. Other measurements are made with counters that record how often some event has happened (e.g., number of lost TPDU's). Finally, one is often interested in knowing the amount of something, such as the number of bytes processed in a certain time interval.

Measuring network performance and parameters has many potential pitfalls. Below we list a few of them. Any systematic attempt to measure network performance should be careful to avoid these.

Make Sure that the Sample Size Is Large Enough

Do not measure the time to send one TPDU, but repeat the measurement, say, one million times and take the average. Having a large sample will reduce the uncertainty in the measured mean and standard deviation. This uncertainty can be computed using standard statistical formulas.

Make Sure that the Samples Are Representative

Ideally, the whole sequence of one million measurements should be repeated at different times of the day and the week to see the effect of different system loads on the measured quantity. Measurements of congestion, for example, are of

little use if they are made at a moment when there is no congestion. Sometimes the results may be counterintuitive at first, such as heavy congestion at 10, 11, 1, and 2 o'clock, but no congestion at noon (when all the users are away at lunch).

Be Careful When Using a Coarse-Grained Clock

Computer clocks work by adding one to some counter at regular intervals. For example, a millisecond timer adds one to a counter every 1 msec. Using such a timer to measure an event that takes less than 1 msec is not impossible, but requires some care.

To measure the time to send a TPDU, for example, the system clock (say, in milliseconds) should be read out when the transport layer code is entered, and again when it is exited. If the true TPDU send time is 300 μ sec, the difference between the two readings will be either 0 or 1, both wrong. However, if the measurement is repeated one million times and the total of all measurements added up and divided by one million, the mean time will be accurate to better than 1 μ sec.

Be Sure that Nothing Unexpected Is Going On during Your Tests

Making measurements on a university system the day some major lab project has to be turned in may give different results than if made the next day. Likewise, if some researcher has decided to run a video conference over your network during your tests, you may get a biased result. It is best to run tests on an idle system and create the entire workload yourself. Even this approach has pitfalls though. While you might think nobody will be using the network at 3 A.M., that might be precisely when the automatic backup program begins copying all the disks to videotape. Furthermore, there might be heavy traffic for your wonderful World Wide Web pages from distant time zones.

Caching Can Wreak Havoc with Measurements

To measure file transfer times, the obvious way to do it is to open a large file, read the whole thing, close it, and see how long it takes. Then repeat the measurement many more times to get a good average. The trouble is, the system may cache the file, so that only the first measurement actually involves network traffic. The rest are just reads from the local cache. The results from such a measurement are essentially worthless (unless you want to measure cache performance).

Often you can get around caching by simply overflowing the cache. For example, if the cache is 10 MB, the test loop could open, read, and close two 10-MB files on each pass, in an attempt to force the cache hit rate to 0. Still, caution is advised unless you are absolutely sure you understand the caching algorithm.

Buffering can have a similar effect. One popular TCP/IP performance utility program has been known to report that UDP can achieve a performance

substantially higher than the physical line allows. How does this occur? A call to UDP normally returns control as soon as the message has been accepted by the kernel and added to the transmission queue. If there is sufficient buffer space, timing 1000 UDP calls does not mean that all the data have been sent. Most of them may still be in the kernel, but the performance utility thinks they have all been transmitted.

Understand What You Are Measuring

When you measure the time to read a remote file, your measurements depend on the network, the operating systems on both the client and server, the particular hardware interface boards used, their drivers, and other factors. If done carefully, you will ultimately discover the file transfer time for the configuration you are using. If your goal is to tune this particular configuration, these measurements are fine.

However, if you are making similar measurements on three different systems in order to choose which network interface board to buy, your results could be thrown off completely by the fact that one of the network drivers is truly awful and is only getting 10 percent of the performance of the board.

Be Careful about Extrapolating the Results

Suppose that you make measurements of something with simulated network loads running from 0 (idle) to 0.4 (40 percent of capacity), as shown by the data points and solid line through them in Fig. 6-47. It may be tempting to extrapolate linearly, as shown by the dotted line. However, many queueing results involve a factor of $1/(1 - \rho)$, where ρ is the load, so the true values may look more like the dashed line.

6.6.3. System Design for Better Performance

Measuring and tinkering can often improve performance considerably, but they cannot substitute for good design in the first place. A poorly designed network can be improved only so much. Beyond that, it has to be redone from scratch.

In this section, we will present some rules of thumb based on experience with many networks. These rules relate to system design, not just network design, since the software and operating system are often more important than the routers and interface boards. Most of these ideas have been common knowledge to network designers for years and have been passed on from generation to generation by word of mouth. They were first stated explicitly by Mogul (1993); our treatment largely parallels his. Another relevant source is (Metcalf, 1993).

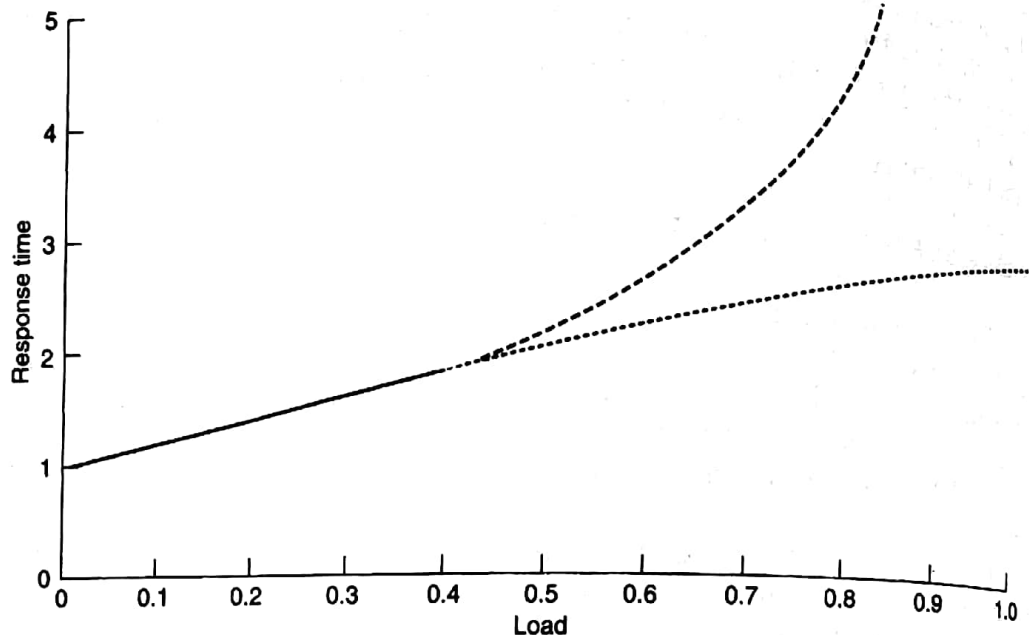


Fig. 6-47. Response as a function of load.

Rule #1: CPU Speed Is More Important than Network Speed

Long experience has shown that in nearly all networks, operating system and protocol overhead dominates actual time on the wire. For example, in theory, the minimum RPC time on an Ethernet is 102 μsec , corresponding to a minimum (64-byte) request followed by a minimum (64-byte) reply. In practice, getting the RPC time down to 1500 μsec is a considerable achievement (Van Renesse et al., 1988). Note that 1500 μsec is 15 times worse than the theoretical minimum. Nearly all the overhead is in the software.

Similarly, the biggest problem in running at 1 Gbps is getting the bits from the user's buffer out onto the fiber fast enough and having the receiving CPU process them as fast as they come in. In short, if you double the CPU speed, you often can come close to doubling the throughput. Doubling the network capacity often has no effect since the bottleneck is generally in the hosts.

Rule #2: Reduce Packet Count to Reduce Software Overhead

Processing a TPDU has a certain amount of overhead per TPDU (e.g., header processing) and a certain amount of processing per byte (e.g., doing the checksum). When sending 1 million bytes, the per-byte overhead is the same no matter what the TPDU size is. However, using 128-byte TDUs means 32 times as much per-TPDU overhead as using 4K TDUs. This overhead adds up fast.

In addition to the TPDU overhead, there is overhead in the lower layers to consider. Each arriving packet causes an interrupt. On a modern RISC processor, each interrupt breaks the CPU pipeline, interferes with the cache, requires a change to the memory management context, and forces a substantial number of CPU registers to be saved. An n -fold reduction in TPDUs sent thus reduces the interrupt and packet overhead by a factor of n .

This observation argues for collecting a substantial amount of data before transmission in order to reduce interrupts at the other side. Nagle's algorithm and Clark's solution to the silly window syndrome are attempts to do precisely this.

Rule #3: Minimize Context Switches

Context switches (e.g., from kernel mode to user mode) are deadly. They have the same bad properties as interrupts, the worst being a long series of initial cache misses. Context switches can be reduced by having the library procedure that sends data do internal buffering until it has a substantial amount of them. Similarly, on the receiving side, small incoming TPDUs should be collected together and passed to the user in one fell swoop instead of individually to minimize context switches.

In the best case, an incoming packet causes a context switch from the current user to the kernel, and then a switch to the receiving process to give it the newly-arrived data. Unfortunately, with many operating systems, additional context switches happen. For example, if the network manager runs as a special process in user space, a packet arrival is likely to cause a context switch from the current user to the kernel, then another one from the kernel to the network manager followed by another one back to the kernel, and finally one from the kernel to the receiving process. This sequence is shown in Fig. 6-48. All these context switches on each packet are very wasteful of CPU time and will have a devastating effect on network performance.

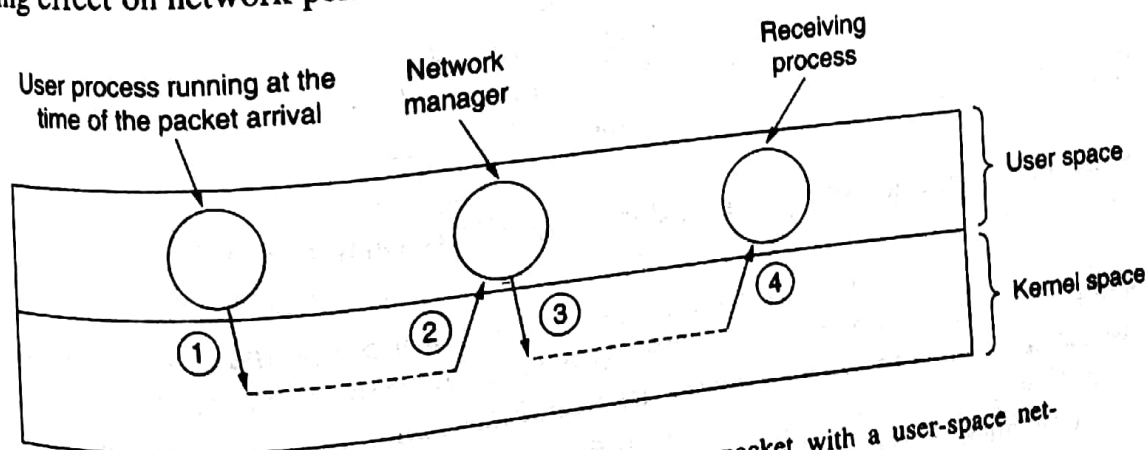


Fig. 6-48. Four context switches to handle one packet with a user-space network manager.

Rule #4: Minimize Copying

Even worse than multiple context switches is making multiple copies. It is not unusual for an incoming packet to be copied three or four times before the TPDU enclosed in it is delivered. After a packet is received by the network interface in a special on-board hardware buffer, it is typically copied to a kernel buffer. From there it is copied to a network layer buffer, then to a transport layer buffer, and finally to the receiving application process.

A clever operating system will copy a word at a time, but it is not unusual to require about five instructions per word (a load, a store, incrementing an index register, a test for end-of-data, and a conditional branch). On a 50-MIPS machine, making three copies of each packet at five instructions per 32-bit word copied requires 75 nsec per incoming byte. Such a machine can thus accept data at a maximum rate of about 107 Mbps. When overhead for header processing, interrupt handling, and context switches is factored in, 50 Mbps might be achievable, and we have not even considered the actual processing of the data. Clearly, handling a 1-Gbps line is out of the question.

In fact, probably a 50-Mbps line is out of the question, too. In the computation above, we have assumed that a 50-MIPS machine can execute any 50 million instructions/sec. In reality, machines can only run at such speeds if they are not referencing memory. Memory operations are often a factor of three slower than register-register instructions, so actually getting 16 Mbps out of the 1 Gbps line might be considered pretty good. Note that hardware assistance will not help here. The problem is too much copying by the operating system.

Rule #5: You Can Buy More Bandwidth but Not Lower Delay

The next three rules deal with communication, rather than protocol processing. The first rule states that if you want more bandwidth, you can just buy it. Putting a second fiber next to the first one doubles the bandwidth but does nothing to reduce the delay. Making the delay shorter requires improving the protocol software, the operating system, or the network interface. Even if all of these are done, the delay will not be reduced if the bottleneck is the transmission time.

Rule #6: Avoiding Congestion Is Better than Recovering from It

The old maxim that an ounce of prevention is worth a pound of cure certainly holds for network congestion. When a network is congested, packets are lost, bandwidth is wasted, useless delays are introduced, and more. Recovering from it takes time and patience. Not having it occur in the first place is better. Congestion avoidance is like getting your DTP vaccination: it hurts a little at the time you get it, but it prevents something that would hurt a lot more.

Rule #7: Avoid Timeouts**56.5**

Timers are necessary in networks, but they should be used sparingly and timeouts should be minimized. When a timer goes off, some action is generally repeated. If it is truly necessary to repeat the action, so be it, but repeating it unnecessarily is wasteful.

The way to avoid extra work is to be careful that timers are set a little bit on the conservative side. A timer that takes too long to expire adds a small amount of extra delay to one connection in the (unlikely) event of a TPDU being lost. A timer that goes off when it should not have uses up scarce CPU time, wastes bandwidth, and puts extra load on perhaps dozens of routers for no good reason.

6.6.4. Fast TPDU Processing

The moral of the story above is that the main obstacle to fast networking is protocol software. In this section we will look at some ways to speed up this software. For more information, see (Clark et al., 1989; Edwards and Muir, 1995; and Chandranmenon and Varghese, 1995).

TPDU processing overhead has two components: overhead per TPDU and overhead per byte. Both must be attacked. The key to fast TPDU processing is to separate out the normal case (one-way data transfer) and handle it specially. Although a sequence of special TPDUs are needed to get into the *ESTABLISHED* state, once there, TPDU processing is straightforward until one side starts to close the connection.

Let us begin by examining the sending side in the *ESTABLISHED* state when there are data to be transmitted. For the sake of clarity, we assume here that the transport entity is in the kernel, although the same ideas apply if it is a user-space process or a library inside the sending process. In Fig. 6-49, the sending process traps into the kernel to do the SEND. The first thing the transport entity does is make a test to see if this is the normal case: the state is *ESTABLISHED*, neither side is trying to close the connection, a regular (i.e., not an out-of-band) full TPDU is being sent, and there is enough window space available at the receiver. If all conditions are met, no further tests are needed and the fast path through the sending transport entity can be taken.

In the normal case, the headers of consecutive data TPDUs are almost the same. To take advantage of this fact, a prototype header is stored within the transport entity. At the start of the fast path, it is copied as fast as possible to a scratch buffer, word by word. Those fields that change from TPDU to TPDU are then overwritten in the buffer. Frequently, these fields are easily derived from state variables, such as the next sequence number. A pointer to the full TPDU header plus a pointer to the user data are then passed to the network layer. Here the same strategy can be followed (not shown in Fig. 6-49). Finally, the network layer gives the resulting packet to the data link layer for transmission.